COLLECTIONS

1. What is Collection Framework?

A Collection Framework is a set of classes and interfaces used to store and manipulate groups of objects.

Hierarchy

```
Iterable
   |
Collection
   |
-----------------------------------------------
|                     |                       |
List                 Set                     Queue
|
ArrayList
LinkedList
Vector

Map (Not part of Collection interface)
|
HashMap
LinkedHashMap
TreeMap
Hashtable
ConcurrentHashMap
```

2. Difference between List, Set and Map

Feature	List	Set	Map
Duplicates	Allowed	Not Allowed	Keys Not Allowed
Order	Maintains	Depends	Depends
Index	Yes	No	No
Example	ArrayList	HashSet	HashMap

Interview Question - Q: When will you use List instead of Set?

Answer:

- Use List when duplicates are allowed.
- Use Set when uniqueness is required.

3. ArrayList vs LinkedList

ArrayList	LinkedList
Dynamic Array	Doubly Linked List
Fast Random Access	Slow Random Access
Slow Insert/Delete in middle	Fast Insert/Delete
Less Memory	More Memory

Internal Structure

ArrayList

```
[10][20][30][40]
```

LinkedList

```
10 <-> 20 <-> 30 <-> 40
```

Interview Question

Which is faster for searching?

ArrayList because of indexing.

```
list.get(5000);
```

Complexity:

```
ArrayList -> O(1)
LinkedList -> O(n)
```

SOME CODING EXAMPLES BELOW –

1. ArrayList Example

Find Duplicate Elements using ArrayList

```
import java.util.*;

public class ArrayListExample {
    public static void main(String[] args) {

        List<String> names = new ArrayList<>();
```

```
        names.add("Java");
        names.add("Spring");
        names.add("Java");
        names.add("Hibernate");
        names.add("Spring");

        Set<String> unique = new HashSet<>();
        Set<String> duplicates = new HashSet<>();

        for(String name : names) {
            if(!unique.add(name)) {
                duplicates.add(name);
            }
        }

        System.out.println("Duplicates: " + duplicates);
    }
}
```

Output

```
Duplicates: [Java, Spring]
```

Interview Point

- Fast random access using index (`get(index)`).
- Backed by dynamic array.
- Insertion/deletion in middle is expensive.

2. LinkedList Example

Insert and Remove Elements Efficiently

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {

        LinkedList<String> tasks = new LinkedList<>();

        tasks.add("Task 1");
        tasks.add("Task 2");
        tasks.add("Task 3");

        tasks.addFirst("Start Task");
        tasks.addLast("End Task");

        System.out.println(tasks);

        tasks.removeFirst();
        tasks.removeLast();

        System.out.println(tasks);
    }
}
```

Output

```
[Start Task, Task 1, Task 2, Task 3, End Task]

[Task 1, Task 2, Task 3]
```

Interview Point

- Doubly linked list implementation.
- Fast insertion/deletion at beginning and end.
- Slow random access (`get(index)`).

3. Vector Example

Thread-Safe Collection

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {

        Vector<Integer> vector = new Vector<>();

        vector.add(10);
        vector.add(20);
        vector.add(30);

        for(Integer num : vector) {
            System.out.println(num);
        }
    }
}
```

Output

```
10
20
30
```

Interview Point

- Synchronized (Thread-safe).
- Slower than ArrayList due to synchronization.
- Legacy collection class.

Frequently Asked Interview Coding Questions

Reverse ArrayList

```
List<Integer> list = Arrays.asList(10, 20, 30, 40);

Collections.reverse(list);

System.out.println(list);
```

Output:

```
[40, 30, 20, 10]
```

Convert ArrayList to LinkedList

```
ArrayList<String> arrayList = new ArrayList<>();
arrayList.add("Java");
arrayList.add("Spring");

LinkedList<String> linkedList = new LinkedList<>(arrayList);

System.out.println(linkedList);
```

Iterate Vector using Enumeration

```
Vector<String> vector = new Vector<>();
vector.add("A");
vector.add("B");
vector.add("C");

Enumeration<String> e = vector.elements();

while(e.hasMoreElements()) {
    System.out.println(e.nextElement());
}
```

Most Asked Difference

Feature	ArrayList	LinkedList	Vector
Data Structure	Dynamic Array	Doubly Linked List	Dynamic Array
Thread Safe	No	No	Yes
Random Access	Fast O(1)	Slow O(n)	Fast O(1)
Insert/Delete Middle	Slow	Fast	Slow
Performance	Best	Good	Slow
Legacy Class	No	No	Yes

One-Line Interview Answer

Use **ArrayList** when reads are more frequent, **LinkedList** when insertions/deletions are frequent, and **Vector** only when built-in synchronization is required.

4. HashSet

Stores unique values.

```
Set<Integer> set = new HashSet<>();

set.add(10);
set.add(10);

System.out.println(set);
```

Output:

```
[10]
```

Duplicate removed.

---

5. HashMap Internal Working (VERY IMPORTANT)

Most asked interview question.

```
Map<Integer,String> map = new HashMap<>();
```

Steps

1. Calculate hashCode()
2. Determine bucket index
3. Store Key-Value pair
4. Handle collision using

Java 7:

```
Linked List
```

Java 8:

```
Linked List
      ↓
Red Black Tree
```

When bucket size > 8.

When an interviewer asks "How does HashMap work internally?", they expect you to explain:

1. Hashing
2. Bucket calculation
3. Collision handling
4. Java 7 vs Java 8 improvements
5. Time complexity
6. Coding example

Step 1: Create HashMap

```
Map<Integer, String> map = new HashMap<>();

map.put(101, "Sudhanshu");
map.put(102, "Amazon");
map.put(103, "Java");
```

Internally HashMap stores data as:

```
Node<K,V>[]
```

Think of it as:

```
Bucket 0 -> null
Bucket 1 -> (101, Sudhanshu)
Bucket 2 -> null
Bucket 3 -> (102, Amazon)
Bucket 4 -> (103, Java)
...
```

Step 2: Hash Calculation

When you execute:

```
map.put(101, "Sudhanshu");
```

HashMap calls:

```
Integer key = 101;

int hash = key.hashCode();
```

For Integer:

```
hashCode() = value itself
```

So:

```
101.hashCode() = 101
```

Step 3: Bucket Index Calculation

HashMap calculates:

```
bucketIndex = hash & (capacity - 1);
```

Default capacity:

```
16
```

Therefore:

```
101 & (16 - 1)

101 & 15

= 5
```

So element is stored in:

```
Bucket 5
```

Visual Representation

```
Index

0 -> null
1 -> null
2 -> null
3 -> null
4 -> null
5 -> (101, Sudhanshu)
6 -> null
7 -> null
...
```

Step 4: Collision Handling

Collision means:

Two keys produce the same bucket index.

Example:

```
map.put(1, "Java");
map.put(17, "Spring");
```

Capacity:

```
16
```

Bucket:

```
1 % 16 = 1

17 % 16 = 1
```

Both go to Bucket 1.

Java 7 Collision Handling

Java 7 stores collided entries as Linked List.

```
Bucket 1

(1, Java)
     ↓
(17, Spring)
     ↓
null
```

Internally:

```
Node
{
   int hash;
   K key;
   V value;
   Node next;
}
```

Example

```
HashMap<Integer,String> map = new HashMap<>();

map.put(1,"Java");
map.put(17,"Spring");

System.out.println(map.get(17));
```

HashMap:

1. Finds bucket
2. Traverses linked list

3. Compares key
4. Returns value

Output:

```
Spring
```

Problem in Java 7

Suppose many collisions occur.

```
Bucket 1

1
↓
17
↓
33
↓
49
↓
65
↓
81
↓
97
↓
113
↓
129
```

Searching:

```
map.get(129);
```

Must traverse entire list.

Complexity:

```
O(n)
```

Slow.

Java 8 Improvement

If bucket size becomes greater than 8:

```
Linked List
      ↓
Red Black Tree
```

This process is called:

```
Treeification
```

Before Treeification

```
1
↓
17
↓
33
↓
49
↓
65
↓
81
↓
97
↓
113
↓
129
```

After Treeification

```
          49
        /   \
      17     81
     / \     / \
    1 33 65 113
             /
           97
```

Now searching becomes faster.

Complexity Comparison

Structure	Search
Linked List	O(n)
Red Black Tree	O(log n)

This is why Java 8 HashMap performs better under heavy collisions.

Treeification Conditions

Tree conversion happens only if:

```
Bucket size > 8
```

AND

```
Table capacity >= 64
```

Otherwise HashMap resizes instead.

Interviewers love this point.

How get() Works Internally

```
map.get(101);
```

Step 1

Calculate hash

```
101.hashCode()
```

Step 2

Calculate bucket

```
hash & (n-1)
```

Step 3

Go directly to bucket

```
Bucket 5
```

Step 4

Compare keys

```
if(node.key.equals(key))
```

Step 5

Return value

```
Sudhanshu
```

How remove() Works

```
map.remove(101);
```

HashMap:

1. Calculate hash
2. Find bucket
3. Find matching key
4. Remove node

Average Complexity:

```
O(1)
```

Custom Object Example (Interview Favorite)

```
class Employee {

    int id;

    Employee(int id){
        this.id=id;
    }

    @Override
    public int hashCode() {
        return id;
    }

    @Override
    public boolean equals(Object obj) {

        Employee e=(Employee)obj;

        return this.id==e.id;
    }
}
```

Using HashMap:

```
HashMap<Employee,String> map = new HashMap<>();

map.put(new Employee(1),"Sudhanshu");
```

```
System.out.println(
    map.get(new Employee(1))
);
```

Output:

```
Sudhanshu
```

Because:

```
hashCode()
```

and

```
equals()
```

are correctly overridden.

What Happens If equals() Is Not Overridden?

```
map.put(new Employee(1),"Sudhanshu");

map.get(new Employee(1));
```

Output:

```
null
```

Because JVM compares:

```
Object references
```

instead of actual id values.

Load Factor

Default Load Factor:

```
0.75
```

Default Capacity:

```
16
```

Resize threshold:

```
16 * 0.75
```

```
= 12
```

When 13th element is inserted:

```
Capacity
```

```
16 → 32
```

HashMap performs:

```
Rehashing
```

and redistributes entries.

Interview Answer (2-Minute Version)

HashMap stores data in an array of buckets. During put(), it calculates the key's hashCode, determines the bucket index using `hash & (capacity - 1)`, and stores the key-value pair. If multiple keys map to the same bucket, a collision occurs. In Java 7 collisions are handled using a Linked List. In Java 8, when the number of nodes in a bucket exceeds 8 and capacity is at least 64, the Linked List is converted into a Red-Black Tree, improving lookup time from O(n) to O(log n). Average complexity for put(), get(), and remove() is O(1). Worst-case complexity is O(n) in Java 7 and O(log n) in Java 8 after treeification.

6. HashMap vs Hashtable

HashMap	Hashtable
Not Synchronized	Synchronized
Faster	Slower
Allows One Null Key	No Null
Thread Unsafe	Thread Safe

7. HashMap vs ConcurrentHashMap

HashMap

Not thread-safe.

```
Map<Integer,String> map = new HashMap<>();
```

ConcurrentHashMap

Thread-safe.

```
Map<Integer,String> map =
      new ConcurrentHashMap<>();
```

Uses:

```
Locking mechanism
CAS Operations
```

Frequently asked in backend interviews.

EXAMPLES –

1. HashMap Problem in Multi-threading

Example

```
import java.util.HashMap;
import java.util.Map;

public class HashMapDemo {

    public static void main(String[] args) throws Exception {

        Map<Integer, String> map = new HashMap<>();

        Thread t1 = new Thread(() -> {

            for(int i=1;i<=1000;i++) {
                map.put(i, "Java");
            }

        });

        Thread t2 = new Thread(() -> {

            for(int i=1001;i<=2000;i++) {
                map.put(i, "Spring");
            }
```

```
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(map.size());
    }
}
```

Expected Output

```
2000
```

Actual Output (May Vary)

```
1945
1987
1993
2000
```

Why?

Because multiple threads are modifying the HashMap simultaneously.

HashMap is **not thread-safe**.

---

2. ConcurrentHashMap Solution

```
import java.util.concurrent.ConcurrentHashMap;

public class ConcurrentHashMapDemo {

    public static void main(String[] args) throws Exception {

        ConcurrentHashMap<Integer, String> map =
                new ConcurrentHashMap<>();

        Thread t1 = new Thread(() -> {

            for(int i=1;i<=1000;i++) {
                map.put(i, "Java");
            }

        });

        Thread t2 = new Thread(() -> {

            for(int i=1001;i<=2000;i++) {
                map.put(i, "Spring");
            }
```

```
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(map.size());
    }
}
```

Output

```
2000
```

ConcurrentHashMap handles concurrent updates safely.

Internal Working

HashMap

```
Thread 1
      ↓
   HashMap
      ↑
Thread 2
```

No synchronization.

Threads can overwrite each other's changes.

ConcurrentHashMap

Java 7

Used **Segment Locking**

```
Segment 1
Segment 2
Segment 3
Segment 4
```

Only the affected segment was locked.

Not the entire map.

Java 8

Segment architecture removed.

Uses:

```
CAS (Compare And Swap)

+
Bucket Level Locking
```

This improves performance significantly.

Null Handling

HashMap

Allowed

```
HashMap<Integer,String> map = new HashMap<>();

map.put(null,"Java");
map.put(1,null);

System.out.println(map);
```

Output

```
{null=Java, 1=null}
```

ConcurrentHashMap

Not Allowed

```
ConcurrentHashMap<Integer,String> map =
        new ConcurrentHashMap<>();

map.put(null,"Java");
```

Output

```
NullPointerException
```

Iteration Behavior

HashMap → Fail Fast

```
HashMap<Integer,String> map = new HashMap<>();

map.put(1,"Java");
map.put(2,"Spring");

for(Integer key : map.keySet()) {

    map.put(3,"Hibernate");

}
```

Output

```
ConcurrentModificationException
```

Because collection was modified while iterating.

ConcurrentHashMap → Weakly Consistent

```
ConcurrentHashMap<Integer,String> map =
        new ConcurrentHashMap<>();

map.put(1,"Java");
map.put(2,"Spring");

for(Integer key : map.keySet()) {

    map.put(3,"Hibernate");

    System.out.println(key);
}
```

Output

```
1
2
```

No exception.

Iterator works on a snapshot-like view.

Atomic Operations (Interview Favorite)

Suppose two threads update a counter.

Wrong Approach

```
map.put("count",
        map.get("count") + 1);
```

Race condition possible.

ConcurrentHashMap Approach

```
ConcurrentHashMap<String,Integer> map =
        new ConcurrentHashMap<>();

map.put("count",0);

map.compute("count",
        (k,v) -> v + 1);
```

Atomic update.

Thread-safe.

---

putIfAbsent()

Very common interview question.

Without ConcurrentHashMap

```
if(!map.containsKey("user")) {
    map.put("user","Sudhanshu");
}
```

Two threads may enter simultaneously.

ConcurrentHashMap

```
map.putIfAbsent(
        "user",
        "Sudhanshu"
);
```

Thread-safe atomic operation.

Real-Time Example

Cache Storage

```
ConcurrentHashMap<String, User> cache =
        new ConcurrentHashMap<>();

cache.put("101", user);
```

Used in:

- Spring Boot applications
- Microservices
- Session management
- API response caching
- Banking applications

Complexity

Operation	HashMap	ConcurrentHashMap
get()	O(1)	O(1)
put()	O(1)	O(1)
remove()	O(1)	O(1)
Thread Safety	No	Yes

Most Asked Interview Questions

Q1: Why not use Hashtable?

Answer:

Hashtable locks the entire table.

```
Hashtable<Integer,String> table =
        new Hashtable<>();
```

Only one thread can work at a time.

ConcurrentHashMap provides finer-grained locking and much better performance.

Q2: Why ConcurrentHashMap doesn't allow null?

Imagine:

```
map.get("user");
```

Returns:

```
null
```

Now we can't determine:

- Key does not exist?
- Value is actually null?

To avoid ambiguity, ConcurrentHashMap prohibits null keys and values.

Q3: When should you use ConcurrentHashMap?

Use ConcurrentHashMap when:

- Multiple threads read/write simultaneously.
- High-performance thread-safe collection is needed.
- Building caches.
- Handling shared application state.

Interview Answer (1 Minute)

HashMap is not thread-safe and should be used in single-threaded environments. ConcurrentHashMap is thread-safe and designed for concurrent access by multiple threads. In Java 8, ConcurrentHashMap uses CAS and bucket-level locking instead of locking the entire map, providing much better performance than Hashtable. HashMap allows one null key and multiple null values, whereas ConcurrentHashMap does not allow null keys or values. Both provide O(1) average complexity for put() and get(), but ConcurrentHashMap ensures thread safety without significantly impacting performance.

8. Comparable vs Comparator

Comparable

Natural Sorting.

```
class Employee implements Comparable<Employee>{

   public int compareTo(Employee e){
      return this.salary - e.salary;
   }
}
```

Comparator

Custom Sorting.

```
Collections.sort(empList,
    (a,b)->a.getName().compareTo(b.getName()));
```

EXAMPLES –

1. Comparable Example

Suppose we want to sort Employees by **ID** (Natural Order).

Employee Class

```
class Employee implements Comparable<Employee> {

    private int id;
    private String name;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int compareTo(Employee e) {

        return this.id - e.id;
    }

    @Override
    public String toString() {
        return id + " " + name;
    }
}
```

Main Class

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<>();

        employees.add(new Employee(3,"Amazon"));
        employees.add(new Employee(1,"Java"));
        employees.add(new Employee(2,"Spring"));

        Collections.sort(employees);

        System.out.println(employees);
    }
}
```

Output

```
[1 Java, 2 Spring, 3 Amazon]
```

Interview Point

```
Collections.sort(employees);
```

works because Employee implements:

```
Comparable<Employee>
```

---

2. Comparator Example

Now sort employees by **Name**.

Main Class

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<>();

        employees.add(new Employee(3,"Amazon"));
        employees.add(new Employee(1,"Java"));
        employees.add(new Employee(2,"Spring"));

        employees.sort(new Comparator<Employee>() {

            @Override
            public int compare(Employee e1,
                               Employee e2) {

                return e1.getName()
```

```
                            .compareTo(e2.getName());
                }
            });

            System.out.println(employees);
        }
}
```

Output

```
[3 Amazon, 1 Java, 2 Spring]
```

Modern Java 8 Comparator

Sort by Name

```
employees.sort(
        (e1,e2) ->
        e1.getName()
          .compareTo(e2.getName())
);
```

Sort by ID

```
employees.sort(
        (e1,e2) ->
        e1.getId() - e2.getId()
);
```

Using Comparator.comparing()

Sort by Name

```
employees.sort(
        Comparator.comparing(Employee::getName)
);
```

Output

```
[3 Amazon, 1 Java, 2 Spring]
```

Sort by Salary (Interview Favorite)

```
class Employee {

    private int id;
    private String name;
```

```
    private double salary;

    // constructor + getters
}
employees.sort(
        Comparator.comparing(
                Employee::getSalary
        )
);
```

Reverse Order Sorting

Comparable

```
Collections.sort(
        employees,
        Collections.reverseOrder()
);
```

Comparator

```
employees.sort(
        Comparator.comparing(Employee::getName)
                  .reversed()
);
```

Output:

```
[Spring, Java, Amazon]
```

Multiple Field Sorting (Very Common)

Sort by:

1. Salary
2. Then Name

```
employees.sort(
        Comparator.comparing(Employee::getSalary)
                  .thenComparing(Employee::getName)
);
```

Example:

```
ID  Name      Salary

1   Sudhanshu 50000
2   Amit      50000
3   Raj       60000
```

Output:

```
Amit
Sudhanshu
Raj
```

String Sorting Example

Comparable Internally

```
List<String> names =
        Arrays.asList(
                "Spring",
                "Java",
                "Hibernate"
        );

Collections.sort(names);

System.out.println(names);
```

Output:

```
[Hibernate, Java, Spring]
```

Because String already implements:

```
Comparable<String>
```

TreeSet Interview Question

```
TreeSet<Employee> employees =
        new TreeSet<>();
```

TreeSet needs ordering.

Therefore either:

Comparable

```
class Employee
implements Comparable<Employee>
```

OR

Comparator

```
TreeSet<Employee> employees =
    new TreeSet<>(
```

```
        Comparator.comparing(Employee::getId)
    );
```

Otherwise:

```
ClassCastException
```

compareTo() Return Values

```
@Override
public int compareTo(Employee e) {

    return this.id - e.id;
}
```

Return Value	Meaning
Negative	Current object smaller
Zero	Equal
Positive	Current object greater

Example:

```
5 - 10 = -5
```

Means:

```
5 comes before 10
```

Most Asked Interview Question

Can we use Comparator without modifying Employee class?

✅ Yes

```
employees.sort(
        Comparator.comparing(Employee::getName)
);
```

This is the biggest advantage of Comparator.

Interview Answer (1 Minute)

Comparable is used for natural sorting and is implemented inside the class using the `compareTo()` method. Comparator is used for custom sorting and is implemented outside the class using the `compare()` method. Comparable supports only one sorting logic, while Comparator supports multiple sorting criteria such as sorting by name, salary, or age. In modern Java, Comparator is often preferred because it provides flexibility through lambda expressions and methods like `Comparator.comparing()` and `thenComparing()`.

9. fail-fast vs fail-safe

Fail Fast

```
ArrayList
HashMap
HashSet
```

Throws:

```
ConcurrentModificationException
```

Example:

```
for(String s:list){
    list.remove(s);
}
```

Fail Safe

```
ConcurrentHashMap
CopyOnWriteArrayList
```

Works on copied collection.

EXAMPLES –

1. Fail-Fast Example

ArrayList

```
import java.util.ArrayList;
import java.util.List;

public class FailFastExample {

    public static void main(String[] args) {

        List<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Spring");
        list.add("Hibernate");

        for(String str : list) {

            if(str.equals("Spring")) {
                list.remove(str);
            }
        }
```

```
    }
}
```

Output

```
Exception in thread "main"
java.util.ConcurrentModificationException
```

Why Exception Occurs?

ArrayList maintains an internal variable:

`modCount`

Whenever collection changes:

```
add()
remove()
clear()
```

`modCount` increases.

Iterator stores:

`expectedModCount`

During iteration:

```
if(modCount != expectedModCount)
{
   throw ConcurrentModificationException
}
```

This behavior is called:

`Fail-Fast`

Meaning:

Fail immediately when collection structure changes.

Internal Flow

```
Iterator Created

expectedModCount = 3

ArrayList Modified
```

```
modCount = 4

expectedModCount != modCount

↓
ConcurrentModificationException
```

Correct Way to Remove

Use Iterator's remove()

```
import java.util.*;

public class Example {

    public static void main(String[] args) {

        List<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Spring");
        list.add("Hibernate");

        Iterator<String> iterator =
                list.iterator();

        while(iterator.hasNext()) {

            String value = iterator.next();

            if(value.equals("Spring")) {
                iterator.remove();
            }
        }

        System.out.println(list);
    }
}
```

Output

```
[Java, Hibernate]
```

No exception.

HashMap Fail-Fast Example

```
HashMap<Integer,String> map =
        new HashMap<>();

map.put(1,"Java");
```

```
map.put(2,"Spring");

for(Integer key : map.keySet()) {

    map.put(3,"Hibernate");
}
```

Output

```
ConcurrentModificationException
```

HashMap iterator is also Fail-Fast.

---

2. Fail-Safe Example

CopyOnWriteArrayList

```
import java.util.concurrent.CopyOnWriteArrayList;

public class FailSafeExample {

    public static void main(String[] args) {

        CopyOnWriteArrayList<String> list =
                new CopyOnWriteArrayList<>();

        list.add("Java");
        list.add("Spring");
        list.add("Hibernate");

        for(String str : list) {

            if(str.equals("Spring")) {
                list.remove(str);
            }
        }

        System.out.println(list);
    }
}
```

Output

```
[Java, Hibernate]
```

No exception.

Why?

CopyOnWriteArrayList creates a copy.

```
Original List

[Java, Spring, Hibernate]

Iterator Works On Snapshot

[Java, Spring, Hibernate]
```

While iteration is happening:

```
list.remove("Spring");
```

modifies a new copy.

Iterator continues on old snapshot.

No exception.

Visualization

```
Current List
--------------------
Java
Spring
Hibernate

Snapshot Used By Iterator
-------------------------
Java
Spring
Hibernate

Remove Spring

New List
--------------------
Java
Hibernate

Iterator Still Uses Snapshot
----------------------------
Java
Spring
Hibernate
```

Thus:

```
No ConcurrentModificationException
```

ConcurrentHashMap Example

```
import java.util.concurrent.ConcurrentHashMap;

public class Example {

    public static void main(String[] args) {

        ConcurrentHashMap<Integer,String> map =
                new ConcurrentHashMap<>();

        map.put(1,"Java");
        map.put(2,"Spring");

        for(Integer key : map.keySet()) {

            map.put(3,"Hibernate");

            System.out.println(key);
        }
    }
}
```

Output

```
1
2
```

No exception.

This is considered Fail-Safe / Weakly Consistent behavior.

Interview Favorite

ArrayList (Fail-Fast)

`ArrayList<String>`

Uses:

`modCount`

Throws:

`ConcurrentModificationException`

CopyOnWriteArrayList (Fail-Safe)

`CopyOnWriteArrayList<String>`

Uses:

```
Snapshot Copy
```

Throws:

```
No Exception
```

Real-Time Use Cases

Fail-Fast

Use when:

- Single-threaded applications
- Want immediate detection of bugs
- Better performance

Examples:

```
ArrayList
HashMap
HashSet
TreeMap
TreeSet
```

Fail-Safe

Use when:

- Multiple threads access collection
- Reads are more frequent than writes
- Concurrent systems

Examples:

```
ConcurrentHashMap
CopyOnWriteArrayList
ConcurrentSkipListMap
```

Interview Answer (30 Seconds)

Fail-Fast iterators work on the original collection and throw `ConcurrentModificationException` if the collection is structurally modified during iteration. Examples are ArrayList, HashMap, and HashSet. Fail-Safe iterators work on a copy or snapshot of the collection, so modifications are allowed during iteration without throwing exceptions. Examples are CopyOnWriteArrayList and ConcurrentHashMap. Fail-Fast is memory efficient and faster, while Fail-Safe provides better support for concurrent environments.

10. Difference Between Iterator and ListIterator

Iterator

```
Iterator<String> itr = list.iterator();
```

Can move:

```
Forward only
```

ListIterator

```
ListIterator<String> itr =
        list.listIterator();
```

Can move:

```
Forward
Backward
Add
Update
Remove
```